

DIAGRAMA DE CLASES

CLASES PRINCIPALES

PAQUETES	CLASE PRINCIPALES	DESCRIPCIÓN
AccesoBD	AccesoBaseDatos	Almacena de forma privada las credenciales de la base de datos
DAO	AdministradorDAO	Centraliza la lógica transaccional el patrón CRUD
Object	MaterialInventario	Añade y valida todo el material que se almacenara en el Inventario
Object	PC	Añade y valida los Pc`s que se almacenará en el Inventario
Object	Ubicación	Asigna una ubicación exacta a cada componente almacenado en el Inventario
Main	App	Se inician los usuarios para su login correcto
Usuarios	Usuario	Clase Objeto que se usa para inicializar los datos básicos de un usuario

RELACIONES

1. Relación: Ubicación y MaterialInventario / PC

- **De Ubicación a Material (1 a 0..N):** Una Ubicación física concreta (por ejemplo, "Almacén Central" o "Aula informática 3") puede estar registrada en el sistema y, en un momento dado, no contener ningún elemento (0) o contener una gran cantidad de ellos (N)
- **De Material a Ubicación (0..1 a 1):** Dependiendo de las reglas de negocio de tu sistema:
 - Si un material obligatoriamente debe estar asignado a una ubicación desde que se da de alta, la relación es **1** (Cada material pertenece estrictamente a 1 ubicación)
 - Si el campo id_ubi de la clase admite valores nulos (por ejemplo, material recién comprado que aún no se ha asignado a ningún aula o estante), la multiplicidad es **0..1** (cero o una ubicación)

2. Relación: Armario / Balda y MaterialInventario

- **De Armario/Balda a Material (1 a 0..N):** Un estante o balda específica (identificada en tu clase por id_balda) puede estar completamente vacía (0) o albergar múltiples materiales y componentes (N)
- **De Material a Armario/Balda (0..1):** Un material puede estar almacenado de forma muy específica en una balda, pero es perfectamente posible que no la tenga (multiplicidad 0) si se trata de un equipo grande (como un servidor en rack o un PC en mesa) o si está en tránsito

3. Relación: PC y Estación de Trabajo / Puesto

- **De Estación a PC (1 a 0..1):** Una estación de trabajo física o un puesto de usuario mapeado en el sistema puede no tener ningún ordenador asignado temporalmente (0) o tener asignado exactamente 1 ordenador (PC)
- **De PC a Estación (1 a 0..1):** Un PC en stock o en el taller de reparación no está asignado a ninguna estación (0), mientras que un ordenador operativo en producción estará vinculado exactamente a 1 puesto de trabajo

4. Relación: AccesoBaseDatos y clases DAO (AdministradorDAO / PcDAO)

- **De AccesoBaseDatos a DAOs (1 a 1..N):** Al utilizarse el patrón **Singleton**, existe única y estrictamente 1 instancia global de la conexión (AccesoBaseDatos). Esta única instancia abastece las consultas de todos los objetos DAO activos (N) que necesiten transaccionar con la base de datos (como mínimo 1 si la aplicación está en uso)
- **De DAO a AccesoBaseDatos (1..N a 1):** Cada vez que AdministradorDAO o PcDAO invocan operaciones CRUD, todos ellos apuntan y consumen los recursos de la misma y única (1) instancia compartida mediante getInstance().

5. Relación: GestionUsuarios y Usuario / Administrador

- **De GestionUsuarios a Usuarios (1 a 0..N):** El gestor o controlador de sesiones maneja un archivo que puede arrancar inicialmente con 0 usuarios registrados en el despliegue del sistema, o ir almacenando un volumen N de cuentas de administradores y operarios a medida que se ejecutan los métodos de registro
- **De Usuario a GestionUsuarios (0..N a 1):** Múltiples instancias de usuarios en memoria (N) son procesadas, leídas y volcadas de manera centralizada hacia 1 único componente de persistencia de archivos de configuración (GestionUsuarios)

DECISIONES DE DISEÑO

A. Desacoplamiento Arquitectónico mediante Patrón DAO e Inversión de Dependencias

El flujo de datos entre la interfaz gráfica (LoginDialog) y el almacenamiento físico no se realiza de forma directa. Se interpone la interfaz RepositorioMaterial<T> y sus implementaciones

- Justificación: Sigue el principio de Inversión de Dependencias (D del principio SOLID). Los módulos de alto nivel (UI) no dependen de los módulos de bajo nivel (sentencias SQL específicas), sino de abstracciones. Esto permite sustituir el motor de base de datos relacional por un ORM como Hibernate, servicios web (API REST) o almacenamiento NoSQL sin alterar una sola línea de código de las pantallas del sistema

B. Integridad de Datos mediante Restricción de Dominio (Tipos Enumerados)

Campos críticos de estado o clasificación como estado en MaterialInventario o categorías en PC han sido tipados utilizando estructuras Enum (Estados, EstadosFungible, Categoria)

- Justificación: Elimina el acoplamiento a cadenas de texto libres (*Magic Strings*), las cuales introducen vulnerabilidades ante errores tipográficos y dificultan la indexación en base de datos. Los enums garantizan seguridad de tipos en tiempo de compilación y aseguran que los estados de los activos físicos del negocio se mantengan homogéneos y normalizados

C. Estrategia de Persistencia Híbrida y Segregación de Responsabilidades

El sistema segrega el almacenamiento: los elementos del inventario se gestionan mediante bases de datos relacionales tradicionales, mientras que el control de acceso se deposita en un archivo estructurado local (ARCHIVO_DAT)

- Justificación: Aplica el Principio de Segregación de Responsabilidades. El inventario demanda alta concurrencia, consultas indexadas complejas, filtros dinámicos y relaciones entre tablas (ventajas nativas de SQL). Por el contrario, la autenticación de usuarios es una operación ligera y lineal que no requiere sobrecargar el servidor de base de datos primario. Aislar las credenciales en un flujo de datos local e independiente simplifica la portabilidad y la seguridad perimetral del sistema de acceso

